Aufgabe 1: Programmierbare Logikbausteine (PLA)

Programmierbare Logikbausteine (PLA) – Formelsammlung & Übersicht

1. Struktur eines PLA (Programmable Logic Array):
Ein PLA realisiert boolesche Schaltnetze zweistufig in disjunktiver Normalform (DNF):

- **Eingänge:** Stehen in wahrer Form ($\mathrm{x_i}$) und invertierter Form ($\neg \mathrm{x_i}$) zur Verfügung.
- **Konjunktionsmatrix (AND):** Programmierbar. Bildet Produktterme $\mathrm{P_k}$ durch wählbare Kreuzungspunkte.
- **Disjunktionsmatrix (OR):** Programmierbar. Verknüpft Produktterme zu Ausgängen $\mathrm{y_j} = \bigvee \mathrm{P_k}$.

2. Berkeley-Espresso-Format (.pla):
`.i <Eingänge> .o <Ausgänge> .p <Produktterme> .e`
In den Produktzeilen: `0` = invertiert ($\neg \mathrm{x}$), `1` = nicht-invertiert (x), `-` = Don't Care (beliebig), gefolgt von der Ausgangsmaske (`1` = aktiv, `0` = inaktiv).

3. Bausteinvergleich:
PROM: AND fest (2^{n}-Decodierer), OR programmierbar | **PAL:** AND programmierbar, OR fest | **PLA:** AND und OR programmierbar (erlaubt Term-Sharing).

Simulator ↗

1.) Espresso-Format zu DNF & Wahrheitstabelle

Gegeben sei folgende PLA-Spezifikation im Berkeley-Espresso-Format mit 4 Eingängen ($\mathrm{x_1}, \mathrm{x_2}, \mathrm{x_3}, \mathrm{x_4}$) und 1 Ausgang ($\mathrm{y}$):

```
.i 4
.o 1
.ilb x1 x2 x3 x4
.ob y
.p 3
000- 1
01-1 1
1--0 1
.e
```

Aufgaben:

1. Geben Sie die boolesche Funktionsgleichung y in disjunktiver Normalform (DNF) an.
2. Bestimmen Sie, für welche Belegungen von $(\mathrm{x_1}, \mathrm{x_2}, \mathrm{x_3}, \mathrm{x_4})$ der Ausgang $\mathrm{y} = 1$ wird.

Simulator ↗

2.) Matrix-Programmierung (AND- & OR-Matrix)

Gegeben seien zwei Funktionen y_1, y_2 mit 3 Eingängen (x_1, x_2, x_3):

$$y_1 = (\neg x_1 \wedge x_2) \vee (x_1 \wedge x_3)$$

$$y_2 = (\neg x_1 \wedge x_2) \vee (\neg x_2 \wedge \neg x_3)$$

Aufgaben:

1. Wie viele Zeilen in der AND-Matrix (Produktterme) werden minimal benötigt?
2. Geben Sie für jeden Produktterm an, welche Eingangsleitungen (x_i bzw. $\neg x_i$) in der AND-Matrix verbunden werden.
3. Beschreiben Sie die Verschaltung der OR-Matrix zur Erzeugung von y_1 und y_2.

Simulator ↗

3.) Produktterm-Sharing zur Hardware-Minimierung

In programmierbaren Logikbausteinen (PLAs) ist die Anzahl der zur Verfügung stehenden Zeilen in der Konjunktionsmatrix (Produktterm-Budget) fest vorgegeben (z. B. maximal 4 oder 8 Produktterme).
Frage: Erklären Sie, warum bei der Minimierung für PLAs eine **gemeinsame Minimierung mehrerer Ausgänge** (Multi-Output Minimization) oft zu einem hardware-effizienteren Ergebnis führt als die getrennte KV-Minimierung jedes einzelnen Ausgangs.

Simulator ↗

4.) Don't-Cares in der Eingabe (BCD-Pseudotetraden)

Ein 4-Bit BCD-Signal ($\mathrm{x_3x_2x_1x_0}$, mit $\mathrm{x_3}$ als MSB) codiert Dezimalziffern von 0 bis 9 ($0000_2 \ldots 1001_2$).
Ein Fehlerdetektor ERR soll eine $\mathtt{1}$ ausgeben, wenn eine unzulässige Pseudotetrade ($10 \ldots 15$) anliegt ($1010_2 \ldots 1111_2$).
Aufgaben:

1. Formulieren Sie die minimierte DNF für ERR unter Ausnutzung von Don't Cares.
2. Geben Sie die entsprechende Spezifikation im Berkeley-Espresso-Format an. Wie viele Produktterme werden im PLA benötigt?

Simulator ↗

5.) Systemvergleich: PROM vs. PAL vs. PLA

In der Digitaltechnik stehen verschiedene Architekturen programmierbarer Bausteine zur Verfügung.

Aufgaben:

1. Beschreiben Sie die Unterschiede bezüglich der Programmierbarkeit von AND- und OR-Matrix bei PROM, PAL und PLA.
2. Warum hat sich in der industriellen Praxis das PAL-Konzept historisch stärker verbreitet als das flexiblere PLA?